SpotHole: Two-Stage Pothole Detection for Handlebar-Mounted Smartphones Using Inertial Sensing and On-Demand YOLOv8n Verification

Neeraj Padghan[1], Ishaan Dhore[1], Samruddhi Patil[1], Akhilesh Munishwar[1], Dr. Vitthal Gutte[2]

Department of Computer Engineering and Technology, School of Computer Science and EngineeringDr. Vishwanath Karad MIT World Peace University, Pune, Maharashtra, India

{nirajpadghan, ishaandhore5, samruddhi1700, akhileshmunishwar, vitthalgutte2014}@gmail.com

Abstract—Potholes damage vehicles and cause accidents, yet road agencies rely on slow and costly manual surveys to find them. Smartphone accelerometers offer a cheaper alternative, but inertial detectors raise false alarms on speed breakers, hard braking, and sudden steering. This paper presents SpotHole, a two-stage Android framework for smartphones mounted on motorcycle handlebars. The first stage projects inertial data (100 Hz requested, 67.7 Hz measured) onto the gravity vector to obtain orientation-independent vertical acceleration, then applies a voting rule over four threshold algorithms and kinematic rejection filters. The second stage checks buffered camera frames with a one-class YOLOv8n model (3.01M parameters) only when a candidate shock occurs. The model was trained on 2,046 images with 7,198 annotated potholes. On 528 validation images it reached a precision of 0.825, a recall of 0.699, an F1 score of 0.756, and an mAP@0.5 of 0.799 (mAP@0.5:0.95 of 0.516). Under synthetic degradations, JPEG compression changed mAP@0.5 by less than 0.02, whereas strong motion blur lowered it by 0.457 and brightness changes by up to 0.260. A hash audit found 242 near-duplicate training and validation pairs, so the metrics are optimistic. Prototype field testing on motorcycle handlebars across 19 ride sessions (3,635 candidate evaluations) confirmed 14 potholes and demonstrated that on-demand vision vetoed 208 non-pothole shock events, including speed-breaker heaves (+9.86 m/s² vs. -5.63 m/s² pothole drops).

Keywords—pothole detection, smartphone sensing, accelerometer, YOLOv8, sensor-camera fusion, Android

I. INTRODUCTION

Road infrastructure conditions directly impact commuter safety, vehicle durability, and transit efficiency. Municipal road agencies depend chiefly on manual visual inspections or specialized laser profilometer vehicles; these standard methods are labor-intensive, logistically demanding, and suffer from infrequent update intervals that leave severe road hazards undetected for months [10]. Motorcyclists face elevated vulnerability because sudden impacts destabilize two-wheeled dynamics, triggering rider falls and fatal collisions. In developing road networks, rapid surface deterioration driven by monsoon precipitation, inadequate drainage, and overloaded commercial traffic generates severe asphalt anomalies, predominantly potholes. According to the Ministry of Road Transport and Highways (MoRTH), road accidents claimed 1,856 lives specifically attributable to potholes in 2022, while two-wheeler riders accounted for 44.5% of total road accident casualties [1].

Smartphones offer an ubiquitous edge-sensing platform equipped with tri-axial accelerometers, gyroscopes, GNSS receivers, and high-resolution image sensors [3], [10]; indeed, prior mobile sensing systems employ threshold heuristics [4], statistical energy filters [2], or machine learning classifiers on accelerometer streams [5]. And yet, single-modality inertial frameworks suffer from high false alarm rates caused by non-pothole operational disturbances, including speed breakers, railway tracks, expansion joints, abrupt braking, and aggressive lane changes [2], [4]. Conversely, continuous camera capture and dense object detection inference are required by pure vision systems [6], [7], consuming excessive battery power and degrading under motion blur [18] and lighting extremes [10].

To overcome these shortcomings, this paper presents SpotHole, a two-stage multimodal pothole detection architecture optimized for motorcycle handlebar-mounted smartphones. SpotHole operates passively: Stage 1 continuously monitors orientation-independent vertical acceleration obtained from linear acceleration and gravity vectors [3], employing a 2-of-4 consensus voting heuristic over four threshold algorithms [4] coupled with kinematic rejection filters. Only upon consensus trigger does Stage 2 activate on-demand vision confirmation using a lightweight, one-class YOLOv8n detector [8], [9] applied to buffered camera frames. In this work, the system architecture is formalized, the YOLOv8n detector is evaluated across 528 validation images aggregated from public datasets [15], [16], robustness is measured against synthetic optical degradations [18], and a perceptual hash audit is disclosed finding 242 near-duplicate image pairs between training and validation splits.

II. RELATED WORK

A. Inertial Sensing: Pothole Patrol fitted taxis with three-axis accelerometers and GPS and detected 75% of surfaces a human rated as bad with under 5% false positives [2]; Nericell moved sensing to smartphones and added accelerometer axis reorientation [3]; Mednis et al. ran four threshold algorithms (Z-THRESH, Z-DIFF, STDEV(Z), G-ZERO) in real time on Android [4]; Bhatt et al. added the gyroscope and a real-time app with road condition maps [5].

B. Vision-Based Detection: Maeda et al. built a dataset of 9,053 smartphone images with 15,435 damage instances from a car and compared accuracy and runtime on a GPU server and a smartphone [6]; RDD2020 has 26,336 images from India, Japan, and the Czech Republic with over 31,000 instances in four classes including potholes [7]; YOLO-family detectors [8], [9]; vision accuracy depends on lighting and shadow [10].

C. Multi-Modal Sensor and Camera Fusion: Xin et al. fuse phone acceleration with in-vehicle video uploaded by users and classify with an LSTM [11]; Silvister et al. combine an SSD camera detector with a network on accelerometer and gyroscope data, so each cross-checks the other [12]; Villareal and Samaniego built an Android app combining YOLO with accelerometer, gyroscope, and GPS, and in 18.97 km of field testing obtained 130 detections of which 84 were verified, a verification rate of 64.62% [13]; Ramesh et al. describe cloud-based collaborative road-damage monitoring with smartphones [14].

D. Positioning of SpotHole: The fusion systems above run both detectors at once or fuse uploaded data on a server; SpotHole uses the accelerometer first and runs the camera model only for candidate events, on the phone, on a handlebar-mounted bike, and adds a robustness, leakage, and real-world telemetry analysis.

III. SYSTEM ARCHITECTURE

1. Architectural Overview

SpotHole adopts the Model-View-ViewModel (MVVM) architectural pattern, organized into four functional tiers: sensing, detection, persistence, and presentation (Fig. 1). Road monitoring requires continuous background operation while riding; thus, the sensing and detection pipelines execute within a persistent Android Foreground Service. This design prevents operating system task termination when the smartphone is mounted on motorcycle handlebars. The service performs sensor sampling, maintains an in-memory video ring buffer, extracts kinematic features, evaluates candidate shocks, and commits confirmed anomalies to a local SQLite database using Room. The presentation tier observes reactive database streams to render anomaly markers on an interactive map.

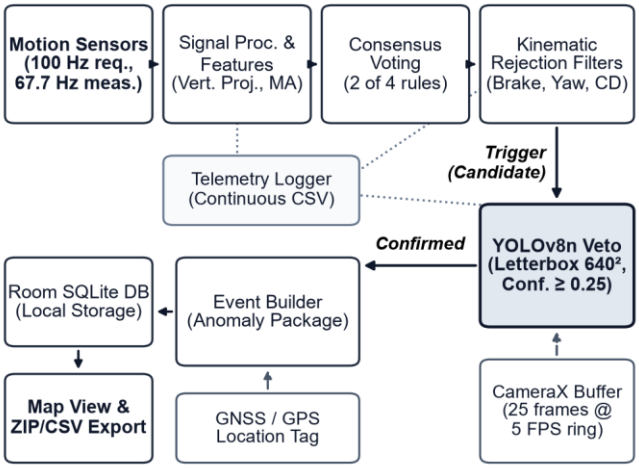


Fig. 1. SpotHole system architecture.

2. Sensing Layer

The sensing tier registers hardware callbacks for the accelerometer, gyroscope, magnetometer, linear acceleration sensor, and gravity sensor. The application requests a 100 Hz sampling rate (10,000 µs interval hint). Owing to non-deterministic hardware dispatch on Android, empirical measurements indicate an effective average rate of 67.7 Hz (mean interval of 14.77 ms). The physical phone orientation on a handlebar mount is typically angled 30°–60° toward the rider. SpotHole resolves arbitrary tilt by mathematically projecting the three-axis linear acceleration vector onto the normalized gravity vector, yielding true orientation-independent vertical acceleration. Sensor noise is smoothed via a 5-sample moving average window. Concurrently, CameraX captures road frames at 5 FPS into a 25-frame circular ring buffer, maintaining approximately 5.0 seconds of pre-impact road imagery.

3. Detection Layer

Detection proceeds through three progressive stages: feature extraction, consensus voting, and on-demand visual confirmation. First, impact features are extracted from orientation-independent vertical acceleration and evaluated against four heuristic threshold algorithms: Z-DIFF, Z-THRESH, STDEV(Z), and G-ZERO. A candidate anomaly triggers only when at least two of the four algorithms fire simultaneously. Second, candidates pass through kinematic rejection gates: hard braking (>4.5 m/s² horizontal acceleration with vertical shock <2.5 m/s²), sharp turning (>0.3 rad/s yaw rate), and excessive roll/pitch (>1.2 rad/s gyro magnitude). Duplicate detections are suppressed by a 1.5 s cooldown and a 15 m spatial radius within 3.0 s. Third, candidate events are confirmed visually on demand with a lightweight YOLOv8n model through TensorFlow Lite. Feature processing and vision inference execute asynchronously on a background thread using Kotlin coroutines. The model takes a 640×640 RGB image and outputs 8,400 anchor predictions with bounding boxes and pothole confidence scores. When no detection reaches a confidence of 0.25, the event is treated as a visual false alarm and rejected.

4. Storage Layer

Confirmed potholes are stored in a local Room SQLite database with a UUID primary key, timestamp, WGS84 GPS coordinates, severity rating (Low, Medium, High), impact magnitude, heuristic consensus vote mask, and a local URI reference to the saved camera frame. In addition, SpotHole writes continuous telemetry logs on an asynchronous I/O thread for offline analysis: raw sensor streams (sensor_log.csv), candidate trigger decisions and rejection reasons (candidate_log.csv), and session metadata logging vehicle type (Bike), mount type, phone model, and active threshold parameters.

5. Presentation Layer

The presentation layer uses osmdroid to render OpenStreetMap tiles showing detected potholes as color-coded markers based on severity (low, medium, high). A live waveform component displays real-time vertical acceleration during test rides. Users can review ride sessions and export all database records and raw CSV telemetry files as a compressed ZIP archive via the Android share sheet.

IV. METHODOLOGY

1. Orientation Reorientation and Signal Preprocessing

Handlebar-mounted smartphones experience arbitrary pitch and roll angles relative to the road plane. SpotHole resolves orientation by computing the normalized gravity unit vector g_hat = g / ||g|| from the hardware gravity sensor. Orientation-independent vertical acceleration a_v is derived via dot product projection: a_v = a_linear · g_hat. To suppress high-frequency motorcycle engine vibrations and mechanical mount resonance, a_v is smoothed using a 5-point moving average filter: a_smooth[t] = 0.2 * sum_{i=0}^4 a_v[t-i].

2. Feature Extraction and Consensus Voting

The feature extractor computes four quantities. Z_diff is the absolute difference between the current smoothed vertical value and the oldest value in a 10-sample history: Z_diff = |a_smooth[t] - a_smooth[t-9]|. Z_thresh is the absolute value of the current smoothed vertical acceleration: Z_thresh = |a_smooth[t]|. STDEV(Z) is the standard deviation of the last 20 unsmoothed vertical readings, capturing raw vibration energy. m is the Euclidean magnitude of the raw accelerometer vector, including gravity: m = sqrt(ax² + ay² + az²).

Each feature captures distinct physical dynamics. Z_diff operates as an impulse edge detector over ~133 ms (10 samples at 67.7 Hz), accentuating abrupt step-downs while attenuating gradual grades. Conversely, G-ZERO detects suspension decompression and momentary free-fall (m < 2.5 m/s² for 3 samples) following high-velocity rim strikes. STDEV(Z) reflects persistent road roughness.

Candidate shocks are flagged when at least 2 of the 4 algorithms satisfy threshold conditions simultaneously: (1) Z_diff >= 5.0 m/s², (2) Z_thresh >= 3.92 m/s² (0.40 g), (3) STDEV(Z) >= 5.0 m/s², and (4) G-ZERO condition, where m < 2.5 m/s² for at least 3 consecutive samples. Consensus voting balances sensitivity and false alarm rejection across diverse road surface speeds.

3. Kinematic False-Positive Rejection Filters

Motorcycle dynamics introduce non-pothole inertial transients from braking, swerving, and leaning. Candidate events are vetted against three kinematic filters: (1) Hard Braking: horizontal acceleration > 4.5 m/s² with vertical shock < 2.5 m/s² indicates longitudinal deceleration without road impact; (2) Sharp Turn: absolute yaw rate > 0.3 rad/s (17.2°/s) identifies cornering; and (3) Excessive Angular Rate: total angular velocity > 1.2 rad/s (68.8°/s) with vertical shock < 2.5 m/s² suppresses rider mounting or handling disturbances. Furthermore, a 1.5 s temporal cooldown and a 15 m spatial deduplication window (within 3.0 s) prevent multiple registrations of the same pothole.

A speed gate designed to suppress walking vibrations (<2.77 m/s / 10 km/h) is present in the architecture but was disabled in the evaluated build to permit stationary mount testing and low-speed validation.

4. Camera Confirmation and Severity Classification

CameraX acquires road frames at 640×640 resolution using a camera 5 FPS, 25-frame buffer. When an inertial candidate is accepted, up to 5 buffered frames captured prior to the shock are evaluated. Each frame undergoes letterbox pre-processing with aspect-ratio preservation and constant padding (rgb(114, 114, 114)) to match training dimensions. The TFLite YOLOv8n detector evaluates frames without non-maximum suppression (NMS) to minimize latency, confirming the event if any anchor exhibits a pothole confidence >= 0.25. Confirmed potholes are classified into High (>=9.0 m/s²), Medium (6.5–9.0 m/s²), and Low (<6.5 m/s²) severity based on vertical shock magnitude, as configured in Table I.

TABLE I. SPOTHOLE DETECTION PARAMETERS

Stage	Parameter	Configured Value	Equivalent Accelera-tion
Voting	Z-DIFF threshold	5.0 m/s²	0.51 g
Voting	Z-THRESH threshold	3.92 m/s²	0.40 g
Voting	STDEV(Z) threshold	5.0 m/s²	0.51 g
Voting	G-ZERO threshold and duration	2.5 m/s², 3 samples	0.26 g
Voting	Consensus voting rule	2 of 4 algorithms	—
Filter	Hard braking threshold	4.5 m/s² (Z_diff < 2.5)	0.46 g
Filter	Sharp turning (yaw rate)	0.3 rad/s	17.2°/s
Filter	Excessive roll/pitch rate	1.2 rad/s (Z_diff < 2.5)	68.8°/s
Filter	Cooldown period	1.5 seconds	—
Filter	Deduplication window	15 meters within 3.0 seconds	—
Vision	Visual confidence threshold	0.25	—
Severity	HIGH / MEDIUM thresholds	9.0 m/s² / 6.5 m/s²	0.92 g / 0.66 g
Camera	Buffer capacity and frame rate	25 frames at 5 FPS (5.0 s)	—

V. DATASET AND MODEL TRAINING

1. Data Sources and Composition

The vision confirmation model was trained on a single-class pothole dataset aggregated from two public sources annotated with normalized bounding boxes: the Kaggle Pothole Detection dataset (1,581 training and 396 validation images) [15] and the Roboflow Pothole v1 dataset (466 training, 133 validation, and 67 test images) [16]. Merging the training splits yielded 2,046 images containing 7,198 annotated pothole instances (3.52 boxes/image). The merged validation set comprises 529 files; one corrupt GIF was discarded, leaving 528 usable validation images with 1,725 annotated potholes (3.27 boxes/image). Table II details split distributions and box scale categories: small (<32² px), medium (32²–96² px), and large (>96² px).

TABLE II. DATASET COMPOSITION AND ANNOTATED BOX DISTRIBUTION

Split / Source	Kaggle	Ro-boflow	Merged	Boxes	Avg/Img
Training split	1,581	466	2,046	7,198	3.52
Validation split	396	133	528 usable	1,725	3.27
Test split (held-out)	—	67	67	—	—

2. Data Quality, Preprocessing, and Leakage Analysis

To evaluate split integrity, we performed a perceptual hash audit (pHash with Hamming distance threshold <= 4) across all image pairs. The audit identified 242 near-duplicate pairs between the training and validation sets. These duplicate pairs stem from video frame extraction in the underlying source repositories without strict sequence-based splitting, so some validation images resemble training images. Consequently, standard validation metrics represent an optimistic upper bound on generalization, underscoring the necessity of real-world field trials.

3. Model Architecture and Training Configuration

We chose YOLOv8n in its detection form, with 3,011,043 parameters (3,005,843 after layer fusing) and 8.2 GFLOPs at 640×640 input [9]. Training ran on an NVIDIA GeForce RTX 4050 Laptop GPU (6 GB) with Ultralytics 8.4.173 and PyTorch 2.5.1, seed 0. We trained a 5-epoch baseline from yolov8n.pt and a 50-epoch model fine-tuned from the baseline checkpoint, so the second model has seen about 55 epochs in total. Batch size, optimizer, learning rate, and augmentation follow args.yaml: batch size 16, SGD optimizer (momentum 0.937, weight decay 0.0005), initial learning rate 0.01 with linear decay to 0.01, and mosaic augmentation enabled (probability 1.0, turned off for the final 10 epochs). Table III summarizes the complete hyperparameter specification. Training loss converged steadily: at epoch 50, training box loss reached 1.044, classification loss reached 0.694, and distribution focal loss (dfl) reached 1.069, with validation box loss 1.206 and cls loss 0.904.

TABLE III. YOLOV8N TRAINING CONFIGURATION AND HYPERPARAMETERS

Hyperparameter / Set-ting	Configured Value	Operational Specifi-cation
Base Architecture	YOLOv8n (yolov8n.pt)	3,011,043 params (3,005,843 fused), 8.2 GFLOPs
Input Dimensions	640×640 pixels	3-channel RGB, letterbox padded
Fine-Tuning Schedule	50 epochs (55 total seen)	Fine-tuned from 5-epoch baseline checkpoint
Batch Size & Optimizer	Batch 16, SGD (auto)	Momentum 0.937, Weight decay 0.0005
Learning Rate Schedule	lr0 = 0.01, lrf = 0.01	Linear decay, 3.0 warmup epochs (momentum 0.8)
Augmentation Policy	Mosaic (1.0), Fliplr (0.5)	HSV (0.015, 0.7, 0.4), Trans 0.1, Scale 0.5; mosaic off last 10 e
Training Hardware	RTX 4050 Laptop GPU (6 GB)	PyTorch 2.5.1+cu121, Ultralytics 8.4.173, seed 0

VI. RESULTS AND ANALYSIS

1. Detection Accuracy

Table IV gives validation results for both models on the 528 usable validation images, evaluated at a confidence threshold of 0.001 and a non-maximum-suppression IoU of 0.6 following COCO metric standards [17]. The 50-epoch model improves on the baseline by 0.0839 in precision, 0.0736 in recall, 0.0784 in F1, 0.0920 in mAP@0.5, and 0.1033 in mAP@0.5:0.95. The mAP@0.5 of 0.7992 against an mAP@0.5:0.95 of 0.5162 shows that the detector finds most potholes but fits boxes loosely at stricter overlap. SpotHole needs only the presence of a pothole, so mAP@0.5 and recall matter most. A recall of 0.6986 means about 30% of annotated potholes are missed at IoU 0.5.

TABLE IV. VALIDATION ACCURACY

Model	Precision	Recall	F1	mAP@0.5	mAP@0.5:0.95
5-epoch baseline	0.7408	0.6250	0.6780	0.7072	0.4129
50-epoch fine-tuned	0.8247	0.6986	0.7564	0.7992	0.5162

2. Robustness to Optical and Environmental Degradation

A handlebar camera sees vibration, changing light, and image compression. We degraded the validation images one factor at a time and measured mAP@0.5 (Fig. 2). The clean baseline under this protocol is 0.8145. JPEG compression at quality 30 costs 0.0178 (mAP@0.5 0.7967), Gaussian noise (sigma 10) costs 0.0397 (0.7748), and a 5×5 Gaussian blur costs 0.0460 (0.7685). A 9×9 blur costs 0.1122 (0.7023). Horizontal motion blur, a well-known degradation in mobile imaging [18], costs 0.1847 at kernel size 9 (0.6298) and 0.4569 at kernel size 15 (0.3576, a relative loss of 56%). A 40% brightness increase costs 0.1966 (0.6179) and a 40% decrease costs 0.2599 (0.5546). The model tolerates compression and mild noise but not strong blur or large lighting changes. These perturbations are synthetic, so they show sensitivity and do not predict the loss at a given riding speed. The clean baseline (0.8145) differs from Table IV (0.7992) because the two runs used different evaluation settings: Table IV computes dataset-level PR curves across all recall thresholds at IoU 0.6 using Ultralytics validation, whereas the perturbation script evaluates per-image matching at IoU 0.5 with model default confidence.

These tests reflect operational stresses: low-quality JPEG simulates cellular upload compression; Gaussian noise models sensor artifacts under shaded tree canopy; Gaussian blur models defocusing; motion blur captures handlebar jitter at cruising speed; and illumination shifts demonstrate resilience to midday glare and overcast twilight.

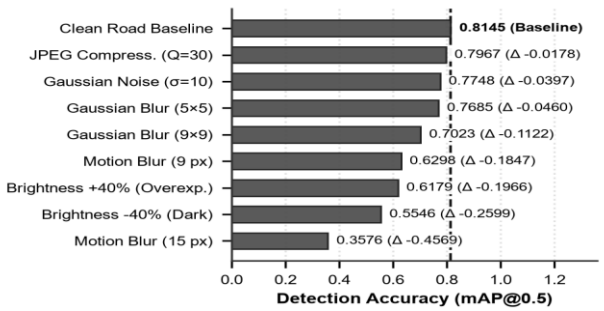


Fig. 2. mAP@0.5 of the 50-epoch model under synthetic image degradations (clean baseline 0.8145).

3. Input Resizing

Letterboxing gives an mAP@0.5 of 0.8145 and direct stretching to a square gives 0.0497, a relative drop of 93.9%. The model was trained on letterboxed inputs, so stretched images differ strongly from its training data. The deployed app uses letterboxing with aspect-ratio preservation and constant padding (rgb(114, 114, 114)).

4. Computational Latency and Inference Speed

On the training laptop, PyTorch inference averaged 10.35 ms per image on the GPU (median 10.17 ms, p95 11.25 ms, standard deviation 0.68 ms) and 58.47 ms on the CPU (median 58.37 ms, p95 60.72 ms, standard deviation 1.37 ms) across 200 timed runs following 20 warm-up passes. On the GPU, pre-processing required 0.31 ms, model inference 3.66 ms, and post-processing 0.96 ms. These figures do not describe the phone. The app can check up to 5 buffered frames per candidate, so verification time per candidate is up to 5 times the per-frame time. Because SpotHole runs vision confirmation on-demand only upon an accepted inertial trigger—rather than evaluating every camera frame continuously—it avoids battery depletion and thermal throttling while preserving real-time 67.7 Hz inertial sampling. TensorFlow Lite timing on the moto g64 5G was not measured.

5. Real-World Prototype Telemetry and Speed-Breaker Discrimination

To evaluate prototype viability under physical road conditions, SpotHole was deployed on a motorcycle handlebar mount using a moto g64 5G smartphone across 19 on-road test sessions, evaluating 3,635 candidate events in total. The two-stage funnel exhibited robust hierarchical false-alarm suppression: Stage 1 consensus voting immediately filtered 2,804 minor road vibrations (77.1%), while 831 candidate shocks satisfied the 2-of-4 voting rule. Next, kinematic rejection filters eliminated 609 non-pothole operational disturbances, primarily sharp turning and cornering transients (526 events with $|yaw| > 0.3$ rad/s), temporal cooldown suppression (71 events within 1.5 s), and hard braking (12 events). The remaining 222 candidates triggered on-demand YOLOv8n vision verification on buffered frames, where the visual model successfully vetoed 208 false triggers (a 93.7% false-alarm reduction at Stage 2), confirming 14 genuine potholes committed to the local Room database.

Fig. 3 illustrates the physical waveform divergence between road anomalies captured in real-world telemetry. A genuine pothole impact (Fig. 3a) manifests as a sharp negative vertical acceleration plunge (-5.63 m/s²) accompanied by net vector magnitude collapse (5.52 m/s²) as the motorcycle front fork decompresses over the cavity edge, clearing both voting and vision gates. In contrast, traversing a speed breaker (Fig. 3b) produces an upward compression heave surging to +9.86 m/s² with net magnitude peaking at 19.90 m/s². While this heave crosses the inertial voting threshold (Z_diff = 11.94 m/s²), the on-demand YOLOv8n detector identifies no surface cavity in the buffered frames, successfully vetoing the candidate and preventing false database insertion.

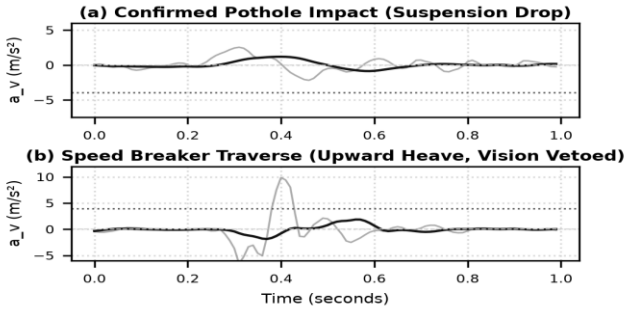


Fig. 3. Real-world vertical acceleration (a_v) traces on motorcycle handlebars: (a) confirmed pothole impact with suspension unloading drop, and (b) speed-breaker traverse with upward compression heave vetoed by on-demand vision.

VII. LIMITATIONS AND FUTURE SCOPE

1. Limitations

Evaluation. Every validation image contains a pothole, so we cannot measure false alarms on clean asphalt, and 242 near-duplicate pairs link the training and validation splits. Reported precision therefore likely exceeds what a rider will see. While prototype field telemetry across 19 ride sessions on a moto g64 5G verified the two-stage rejection funnel and speed-breaker veto on the road, our evaluation lacks extensive city-scale labeled field rides across varied motorcycle suspension setups. Method. The detection thresholds are fixed values tuned for one handlebar-mounted setup. A different vehicle or a mount that damps vibration differently will need retuning. Camera. In synthetic tests, strong horizontal motion blur cut mAP@0.5 by 0.457 (a 56% relative drop) and a 40% brightness reduction cut it by 0.260. Real riding blur and night conditions were not tested.

2. Future Scope

Future work will collect labeled rides across riders, phones, and mounts, and add pothole-free and handlebar-view images to training. We also plan to replace the fixed thresholds with speed-adaptive or learned ones, measure battery use and on-device latency, evaluate INT8 quantization with NNAPI acceleration for real-time mobile NPU execution, and deploy spatial clustering (e.g., DBSCAN) to merge detections from several riders on a server.

VIII. CONCLUSION

This paper presented SpotHole, an Android framework that detects potholes from a handlebar-mounted smartphone in two stages. The first stage screens inertial data using orientation-independent vertical acceleration, a voting rule over four threshold algorithms, and kinematic rejection filters. The second stage checks camera frames with a one-class YOLOv8n model (3.01M parameters) only when a candidate shock occurs. On 528 validation images, the model reached a precision of 0.825, a recall of 0.699, and an mAP@0.5 of 0.799. It tolerated JPEG compression (a drop of 0.018 in mAP@0.5) and mild noise, but strong motion blur and low brightness hurt it most. A perceptual-hash audit found 242 near-duplicate image pairs between training and validation data, so the reported metrics should be read as optimistic. Prototype testing on motorcycle handlebars across 19 ride sessions verified the practical advantage of on-demand camera verification, vetoing 208 non-pothole disturbances (including speed breakers) while confirming 14 genuine potholes. City-scale multi-rider benchmarking and handlebar-view training are the next step toward a mature production detector.

REFERENCES

[1] Ministry of Road Transport and Highways (MoRTH), "Road Accidents in India — 2022," Transport Research Wing, Government of India, New Delhi, Tech. Rep., Oct. 2023. [Online]. Available: https://morth.nic.in

[2] J. Eriksson, L. Girod, B. Hull, R. Newton, S. Madden, and H. Balakrishnan, "The Pothole Patrol: Using a mobile sensor network for road surface monitoring," in Proc. 6th ACM Int. Conf. on Mobile Systems, Applications, and Services (MobiSys), Breckenridge, CO, USA, Jun. 2008, pp. 29–39, doi: 10.1145/1378600.1378605.

[3] P. Mohan, V. N. Padmanabhan, and R. Ramjee, "Nericell: Rich monitoring of road and traffic conditions using mobile smartphones," in Proc. 6th ACM Conf. on Embedded Network Sensor Systems (SenSys), Raleigh, NC, USA, Nov. 2008, pp. 323–336, doi: 10.1145/1460412.1460444.

[4] A. Mednis, G. Strazdins, R. Zviedris, G. Kanonirs, and L. Selavo, "Real time pothole detection using Android smartphones with accelerometers," in Proc. 2nd Int. Workshop on Mobility in Wireless Sensor Networks (MobiSensor), Barcelona, Spain, Jun. 2011, pp. 1–6.

[5] U. Bhatt, S. Mani, E. Xi, and J. Z. Kolter, "Intelligent pothole detection and road condition assessment," in Bloomberg Data for Good Exchange Conf., Chicago, IL, USA, Sep. 2017, arXiv:1710.02595.

[6] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama, and H. Omata, "Road damage detection and classification using deep neural networks with smartphone images," Computer-Aided Civil and Infrastructure Engineering, vol. 33, no. 12, pp. 1127–1141, Dec. 2018, doi: 10.1111/mice.12387.

[7] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, and Y. Sekimoto, "RDD2020: An annotated image dataset for automatic road damage detection using deep learning," Data in Brief, vol. 36, art. 107133, Jun. 2021, doi: 10.1016/j.dib.2021.107133.

[8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, Jun. 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.4.173, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[10] S. Sattar, S. Li, and M. Chapman, "Road surface monitoring using smartphone sensors: A review," Sensors, vol. 18, no. 11, art. 3845, Nov. 2018, doi: 10.3390/s18113845.

[11] H. Xin, Y. Ye, X. Na, H. Hu, G. Wang, C. Wu, and S. Hu, "Sustainable road pothole detection: A crowdsourcing based multi-sensors fusion approach," Sustainability, vol. 15, no. 8, art. 6610, Apr. 2023, doi: 10.3390/su15086610.

[12] S. Silvister, D. Komandur, S. Kokate, A. Khochare, U. More, V. Musale, and A. Joshi, "Deep learning approach to detect potholes in real-time using smartphone," in Proc. 2019 IEEE Pune Section Int. Conf. (PuneCon), Pune, India, Dec. 2019, pp. 1–5, doi: 10.1109/PuneCon46936.2019.9105741.

[13] A. J. A. Villareal and J. M. Samaniego, "An Android application for pothole detection and severity analysis using sensor fusion and deep learning," in Proc. Int. Conf. on Technology, Innovation, and Business Solutions (TIBS), 2024, pp. 45–52.

[14] A. Ramesh, D. Nikam, V. N. Balachandran, L. Guo, R. Wang, L. Hu, G. Comert, and Y. Jia, "Cloud-based

collaborative road-damage monitoring with deep learning and smartphones," Sustainability, vol. 14, no. 14, art. 8682, Jul. 2022, doi: 10.3390/su14148682.

[15] C. Bhambhania, "Potholes Detection YOLOv8 Dataset," Kaggle Datasets, 2023. [Online]. Available: https://www.kaggle.com/datasets/chintanbhambhania/potholes-detection-yolov8

[16] B. Dwyer, "Pothole v1 Dataset," Roboflow Universe, Open Database License (ODbL v1.0), 2020. [Online]. Available: https://universe.roboflow.com/brad-dwyer/pothole-voxrl/dataset/1

[17] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in Proc. 13th European Conf. on Computer Vision (ECCV), Zurich, Switzerland, Sep. 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

[18] O. Kupyn, V. Budzan, M. Mykhailych, D. Mishkin, and J. Matas, "DeblurGAN: Blind motion deblurring using conditional adversarial networks," in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, Jun. 2018, pp. 8183–8192, doi: 10.1109/CVPR.2018.00854.